Wednesday

Started thinking about the challenge. First ideas/questions:

- How to create test vectors, is there a decent library I can use or do I first code it naively?
- What are the range of the objects I will manipulate?
    - Code will be vastly different if I optimize for multiplying large polynomials versus multiplying a large number of polynomials.
    - Complex coefficients **(ai + b)** – what's the range of **a** and **b**? 16 vs 32 vs 64 bits could be the difference between storing in register vs global.
- How do I reuse code? I want the Add and Mul functions to be easily reusable in the context of matrix multiply – And since we want to multiply large matrices, I will need to store things in global memory. How can I reuse code for both cases while keeping coalesced memory accesses?

Thursday

Work done:

- No code today, just reading, planning and pre-thinking about how I'll tackle things.
- Planning the week ahead
- Create the repo on GitHub.

Decisions:

- I'll code the CPU part in Java so that I don't spend too much time with boilerplate (I have some code ready for CUDA integration)
- I'll set the range for the complex coefficients to float 32bits.
- Since the challenge hints at polynomials being of a degree 2^n, I'll assume we are more interested in multiplying large polynomials, rather than a lot of small ones. My code will be optimized accordingly.
- I also assume the 2^n degree is a strong hint at FFT based multiplication of polynomials and since this is the best way I found to do it, I'll implement it this way.

Planning:

- Thursday: Create base project and create a few test vectors, CPU code, no CUDA yet. Read up a bit about polynomial multiplication on GPU.
- Friday: Initial implementations of Add(P1, P2) + Tests / Profiling
- Saturday: Initial implementations of Mul(P1, P2) + Tests / Profiling
- Sunday: implementation of Add(P, Q) + Tests / Profiling
- Monday: implementation of Mul(P, Q) + Tests / Profiling
- Tuesday / Wednesday: packaging, doc and release.

Friday

Implemented Add (P1, P2) kernel and associated CPU code. Nothing fancy, it works. Not sure the addition is worth the memory round trip to/from GPU though.

I did not implement slicing for this kernel, so we're limited to polynomials smaller than 2Gb. This limitation is due to how the java library I'm using is handling memory. I think that if we wanted to handle polynomials this big, we would probably slice the copy and execution in smaller chunks anyway, so not really an issue.

Added unit tests for the addition of polynomials size 1 to 2^15. All is green.

Saturday

Implemented Mul (P1, P2) kernel and associated CPU code. I had a couple kernels to implement and I already regret the CPU representation I chose for the objects (A buffer with all real coefficients and a buffer with all imaginary coefficients). Fortunately, this only impacts the first and last kernel of the process, so I should be able to change it easily later on.

I initially populated the FFT input kernel by recursing the FFT on CPU to populate items at the correct position. This was painfully slow so I replaced it to a straight copy to GPU followed by a kernel moving things in their correct spot.

A couple indexing bugs on the FFT kernel, but quickly solved though tracing and computing the FFT of a 0 1 2 3 4 5 6 7 buffer. Quickly coded a CPU version to debug easily.

Added unit tests for the multiplication of polynomials size 1 to 2^15. All is green.

Profiling: the FFT preparation kernel relocating items is the most time-consuming part. I put it on the list of things to investigate.

Sunday

Life got in the way, no coding today!

Monday

Implement Add(P, Q) CPU code. It's quite easy, but since volumes are bigger now, I need to have some slicing mechanism, since matrices can be very large. It's more about CPU code than GPU code here, since the kernel is quite agnostic to how it's being called. So much agnostic actually that I simply reused the kernel from Add(P1, P2) with zero modification. We're really just summing two very large arrays.

Most of the time is spent on creating the two buffers and copying them, so I added a little threading to do both in parallel.

Added unit tests for the addition of matrices of various sizes and shapes. All is green.

Tuesday

Implemented CPU and GPU code for Mul(P, Q) today. This was a bit more challenging and clearly took me more time than I thought it would. I coded the slicing directly, and this was a mistake. I should have started by implementing the vanilla multiplication, and then slice. I was too greedy.

I also threaded copy and execution of the slices, which had nice benefits (~20% speedup on a 1024 * 1024 matrix of degree 1024 polynomials)

Added unit tests for the multiplication of matrices of various sizes and shapes. All is green.

Profiling: a lot of time is spent on the multiplication kernel. That's to be expected because we're running *height(A) * width(B) * n* threads summing *width(A)* polynomial points. Coalescence seems fine, but definitely a hotspot to verify for optimization. A sum reduction comes to mind.

Wednesday

Initial version is finished and tested, so today was mainly cleanup, documentation, packaging, Readme, etc…